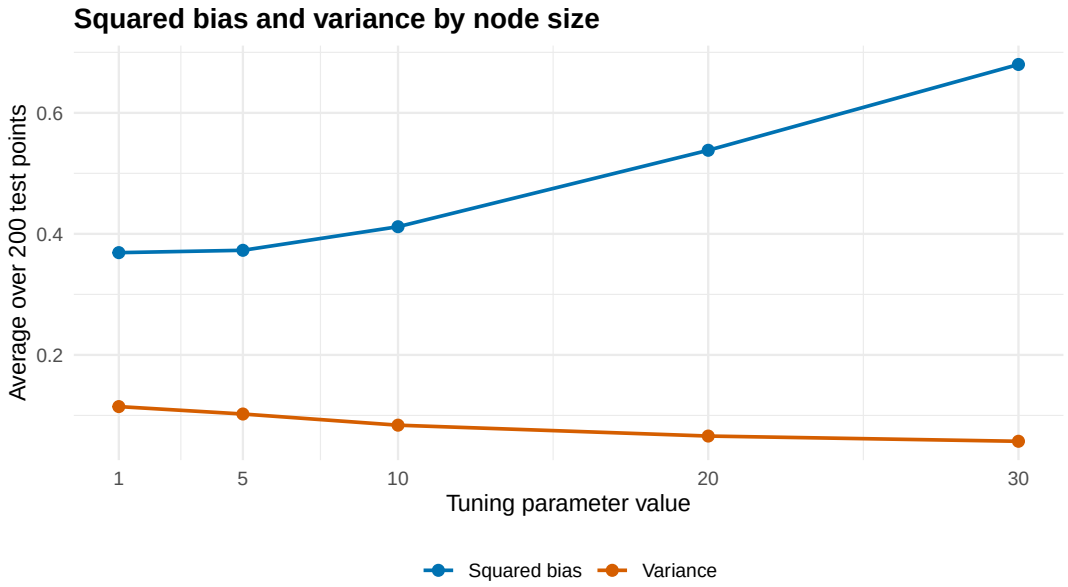
Homework 10: Random Forests

Question 1: Bias-variance trade-off simulation

The simulation used 100 training sets of 200 observations and the same 200 test predictors throughout. Predictors were jointly normal with pairwise correlation 0.3; $Y = X_1 + X_3 + X_5 + \epsilon$, with independent standard normal noise. Each forest had 500 trees, each fit sampled 200 observations with replacement, and all settings within each repetition used the same training data. Seed: 43210. Squared bias was estimated at each test point from the squared difference between the average prediction and the true function; variance was estimated across repetitions. Both are averaged over 200 test points.

Varying node size with $mtry = 3$

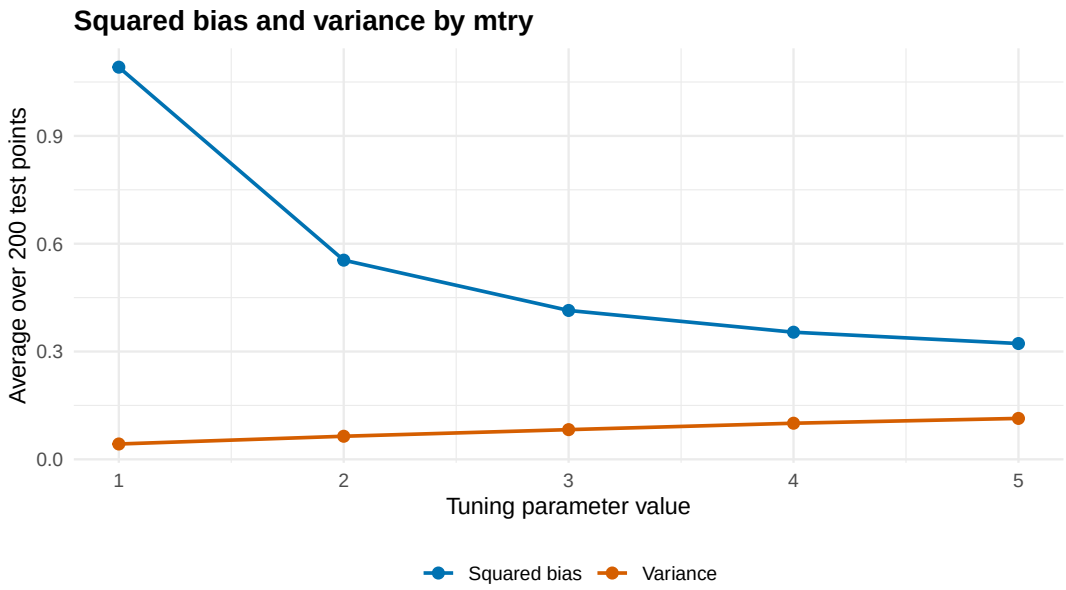
Node size	Averaged squared bias	Averaged variance
1	0.3690	0.1146
5	0.3729	0.1023
10	0.4118	0.0839
20	0.5382	0.0659
30	0.6800	0.0573



Node size 30 substantially smooths the fit: squared bias rises from 0.369 at size 1 to 0.680, while variance falls from 0.115 to 0.057. Because the true regression function is additive and smooth, large leaves underfit its variation. Among tested sizes, size 5 has the smallest estimated squared-bias-plus-variance total (0.475).

Varying $mtry$ with node size 10

$mtry$	Averaged squared bias	Averaged variance
1	1.0913	0.0427
2	0.5542	0.0642
3	0.4143	0.0826
4	0.3537	0.1005
5	0.3222	0.1139



Increasing *mtry* makes informative predictors more likely to be available at a split, reducing squared bias from 1.091 at $mtry = 1$ to 0.322 at $mtry = 5$. The trade-off is greater variance (0.043 to 0.114), as trees use more similar candidate sets and become less diverse. $mtry = 5$ had the smallest estimated squared-bias-plus-variance among those tested; the variance cost may dominate in other datasets. With 100 rather than 200 bootstrap draws, I would initially expect a shift toward smaller node sizes to preserve comparable tree resolution: the expected number of distinct observations is about 63 rather than 127. However, smaller samples also make trees noisier, which can favor larger leaves. This direction is a hypothesis to check by simulation, not a guaranteed result.

Question 2: Tuning random forests for MNIST

The first 1,000 observations supplied training data and the remaining 1,000 supplied held-out test data; only digits 1, 6, and 7 were kept. PCA was centered but not scaled and fitted only to training pixels. The training set had 327 observations (1: 116, 6: 94, 7: 117); the test set had 317 observations (1: 104, 6: 106, 7: 107). Each forest had 1,000 trees and bootstrap samples of the training-set size. Seed: 43211.

Default model and OOB tuning

The default model used $mtry = 6$ and node size 5; its OOB misclassification rate was **0.1525**. The grid minimum was $mtry = 20$ and node size 5, with OOB error **0.0504**. OOB error was used for selection.

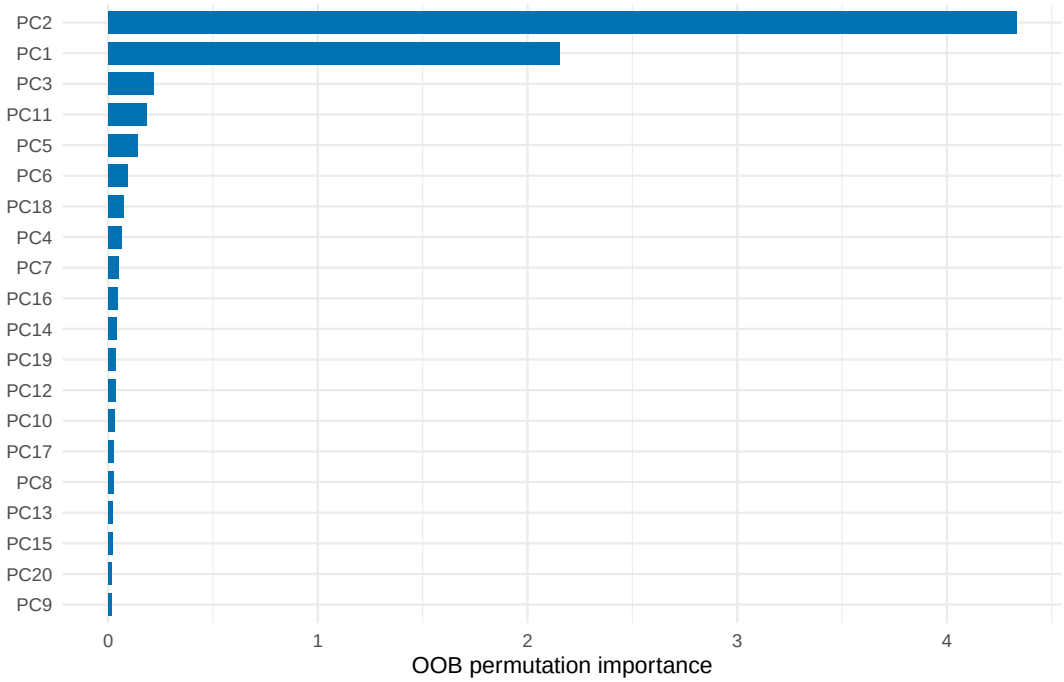
Table 1: OOB misclassification rate for the tuning grid.

mtry	Node size	OOB error	Selected
2	1	0.2857	
4	1	0.1789	
6	1	0.2258	
10	1	0.1240	
15	1	0.2321	
20	1	0.0565	
2	5	0.2500	
4	5	0.2712	
6	5	0.2232	
10	5	0.1083	
15	5	0.0714	
20	5	0.0504	Yes

mtry	Node size	OOB error	Selected
2	10	0.1913	
4	10	0.2787	
6	10	0.2288	
10	10	0.1062	
15	10	0.0738	
20	10	0.1271	
2	20	0.3125	
4	20	0.3058	
6	20	0.1094	
10	20	0.1920	
15	20	0.0960	
20	20	0.1774	
2	30	0.3058	
4	30	0.3030	
6	30	0.0678	
10	30	0.2937	
15	30	0.1301	
20	30	0.1074	

Selected-model permutation importance

RLT's OOB permutation importance is reported for all 20 principal components; larger values indicate greater importance under this measure. PC2 and PC1 are the most influential by a wide margin. Principal components combine pixels, so their importance does not identify individual image pixels.



Component	OOB permutation importance
PC2	4.3308
PC1	2.1512
PC3	0.2187
PC11	0.1826
PC5	0.1398
PC6	0.0929
PC18	0.0748
PC4	0.0641
PC7	0.0508
PC16	0.0454
PC14	0.0406
PC19	0.0379
PC12	0.0363
PC10	0.0321
PC17	0.0258
PC8	0.0254
PC13	0.0198
PC15	0.0191
PC20	0.0189
PC9	0.0168

Held-out test assessment

The selected forest misclassified 5 of 317 test observations, a misclassification rate of **0.0158**. Test labels were not used to fit PCA or select the model. Nearly all held-out digits were classified correctly; digit 7 had the most mistakes.

Actual	Predicted 1	Predicted 6	Predicted 7
1	103	0	1
6	1	105	0
7	0	3	104

Question 3: Planning a random-forest analysis skill

(a) Skill design

I would create a brief `SKILL.md` with YAML frontmatter containing a short name and description, followed by a focused instruction body for future, unspecified datasets. I would not restate routine random-forest knowledge that a capable AI can apply reliably; instead, I would emphasize personal workflow and structural preferences, while retaining essential reproducibility and data-suitability checks. The skill would specify a dataset-aware tuning grid, prefer OOB evaluation or state when cross-validation is appropriate, and set strict table, figure, and report requirements. It would remain under 50 lines, counting blank lines, so its guidance remains concise.

(b) Ten instructions, from basic to personal

1. Set and report a fixed random seed before any random sampling or fitting.
2. Verify response type, predictor encoding, dimensions, missingness, and other data requirements before fitting; explain any required changes.
3. Fit a baseline random forest with 500 trees and the package's default *mtry* and node-size settings.
4. Define and report an explicit, data-appropriate tuning grid, focusing on *mtry* and node size (minimum leaf size).
5. Use OOB error as the primary tuning criterion; use repeated cross-validation when OOB is unsuitable, and exclude test data from model selection.

6. Select task-appropriate metrics: for classification, report OOB/test error and a confusion matrix (and ROC-AUC when appropriate); for regression, report RMSE and R^2.
7. Include a compact, clearly labeled table comparing every tuning setting and its validation performance, with the selected setting marked.
8. Include one publication-ready diagnostic figure, such as OOB error versus the number of trees or a variable-importance plot, with labeled axes.
9. Interpret predictive performance, limitations, and the top three to five influential predictors without overstating importance.
10. Keep the final report to one printed page at 11-point font; prioritize concise evidence, consistent table/figure formatting, and no filler.

Appendix: Reproducible R code

The script generates the simulations, tuning results, importance estimates, figures, and this report. It expects the compressed MNIST file in the homework folder:
`data/mnist-first2000.csv.gz`.

```
#!/usr/bin/env Rscript

suppressPackageStartupMessages({
  library(RLT)
  library(ggplot2)
})

script_arg <- grep("^--file=", commandArgs(trailingOnly = FALSE), value = TRUE)
if (length(script_arg) != 1L) {
  stop("Run this file with Rscript so its location can be identified.")
}
script_file <- normalizePath(sub("^--file=", "", script_arg))
output_dir <- dirname(script_file)
data_file <- file.path(output_dir, "data", "mnist-first2000.csv.gz")

if (!file.exists(data_file)) {
  stop(
    "MNIST data not found. Extract mnist-first2000.csv.gz from the ",
    "STAT 432 week-10 combined ZIP into homework/week-10/data/."
  )
}

q1_seed <- 43210L
q1_repetitions <- 100L
q1_test_size <- 200L
q1_training_size <- 200L
q1_tree_count <- 500L

set.seed(q1_seed)
correlation <- 0.3
predictor_count <- 5L
covariance <- matrix(correlation, predictor_count, predictor_count)
diag(covariance) <- 1
generate_x <- function(n) {
  matrix(rnorm(n * predictor_count), nrow = n) %*% chol(covariance)
}
true_function <- function(x) {
  x[, 1] + x[, 3] + x[, 5]
}

test_x <- generate_x(q1_test_size)
test_truth <- true_function(test_x)
node_sizes <- c(1L, 5L, 10L, 20L, 30L)
mtry_values <- 1:5
node_predictions <- array(
  NA_real_,
  dim = c(q1_test_size, q1_repetitions, length(node_sizes))
)
mtry_predictions <- array(
  NA_real_,
  dim = c(q1_test_size, q1_repetitions, length(mtry_values))
)

cat("Question 1: running the 100-repetition simulation.\n")
for (repetition in seq_len(q1_repetitions)) {
  train_x <- generate_x(q1_training_size)
  train_y <- true_function(train_x) + rnorm(q1_training_size)

  for (setting in seq_along(node_sizes)) {
    fit <- RLT(
      x = train_x,
      y = train_y,
      model = "regression",
      ntrees = q1_tree_count,
      mtry = 3L,
      nmin = node_sizes[setting],
      resample.prob = 1,
      replacement = TRUE,
      reinforcement = FALSE
    )
    node_predictions[, repetition, setting] <-
      predict(fit, test_x)$Prediction
  }

  for (setting in seq_along(mtry_values)) {
    fit <- RLT(
      x = train_x,
      y = train_y,
      model = "regression",
      ntrees = q1_tree_count,
      mtry = mtry_values[setting],
      nmin = 10L,
      resample.prob = 1,
      replacement = TRUE,
      reinforcement = FALSE
    )
    mtry_predictions[, repetition, setting] <-
      predict(fit, test_x)$Prediction
  }

  if (repetition %% 10L == 0L) {
    cat("  completed repetition", repetition, "of", q1_repetitions, "\n")
  }
}

summarize_predictions <- function(predictions, parameter_name, values) {
  rows <- lapply(seq_along(values), function(setting) {
    setting_predictions <- predictions[, , setting, drop = FALSE][, , 1]
    data.frame(
      parameter = parameter_name,
      value = values[setting],
      squared_bias = mean((rowMeans(setting_predictions) - test_truth)^2),
      variance = mean(apply(setting_predictions, 1, var))
    )
  })
```

```
  do.call(rbind, rows)
}

q1_results <- rbind(
  summarize_predictions(node_predictions, "Node size (mtry = 3)", node_sizes),
  summarize_predictions(mtry_predictions, "mtry (node size = 10)", mtry_values)
)
write.csv(
  q1_results,
  file.path(output_dir, "q1-bias-variance.csv"),
  row.names = FALSE
)

plot_bias_variance <- function(data, title, output_file) {
  long_data <- rbind(
    data.frame(value = data$value, metric = "Squared bias", estimate = data$squared_bias),
    data.frame(value = data$value, metric = "Variance", estimate = data$variance)
  )
  plot <- ggplot(
    long_data,
    aes(x = value, y = estimate, color = metric, group = metric)
  ) +
    geom_line(linewidth = 0.8) +
    geom_point(size = 2.2) +
    scale_color_manual(values = c("Squared bias" = "#0072B2", "Variance" = "#D55E00")) +
    scale_x_continuous(breaks = sort(unique(data$value))) +
    labs(title = title, x = "Tuning parameter value", y = "Average over 200 test points",
         color = NULL) +
    theme_minimal(base_size = 11) +
    theme(legend.position = "bottom", plot.title = element_text(face = "bold"))
  ggsave(file.path(output_dir, output_file), plot, width = 7, height = 4, units = "in")
}

plot_bias_variance(
  q1_results[q1_results$parameter == "Node size (mtry = 3)", ],
  "Squared bias and variance by node size",
  "q1-node-size.pdf"
)
plot_bias_variance(
  q1_results[q1_results$parameter == "mtry (node size = 10)", ],
  "Squared bias and variance by mtry",
  "q1-mtry.pdf"
)

mnist <- read.csv(gzfile(data_file), check.names = FALSE)
if (nrow(mnist) != 2000L || ncol(mnist) != 785L ||
    !identical(names(mnist)[1], "Digit") || anyNA(mnist)) {
  stop("The MNIST file must contain 2,000 complete rows and Digit plus 784 pixels.")
}
if (!all(mnist$Digit %in% 0:9)) {
  stop("Digit must contain integer labels from 0 through 9.")
}

train_rows <- seq_len(1000L)
test_rows <- 1001:2000
selected_digits <- c(1L, 6L, 7L)
train_keep <- train_rows[mnist$Digit[train_rows] %in% selected_digits]
test_keep <- test_rows[mnist$Digit[test_rows] %in% selected_digits]
pixel_names <- names(mnist)[-1]
train_pixels <- as.matrix(mnist[train_keep, pixel_names])
test_pixels <- as.matrix(mnist[test_keep, pixel_names])
train_labels <- factor(mnist$Digit[train_keep], levels = selected_digits)
test_labels <- factor(mnist$Digit[test_keep], levels = selected_digits)

pca <- prcomp(train_pixels, center = TRUE, scale. = FALSE, rank. = 20L)
train_pcs <- pca$x[, seq_len(20L), drop = FALSE]
test_pcs <- predict(pca, newdata = test_pixels)[, seq_len(20L), drop = FALSE]
colnames(train_pcs) <- paste0("PC", seq_len(20L))
colnames(test_pcs) <- colnames(train_pcs)

q2_seed <- 43211L
q2_tree_count <- 1000L
set.seed(q2_seed)
cat("Question 2: fitting the default random forest.\n")
default_fit <- RLT(
  x = train_pcs,
  y = train_labels,
  model = "classification",
  ntrees = q2_tree_count,
  resample.prob = 1,
  replacement = TRUE,
  reinforcement = FALSE
)
default_oob_error <- tail(default_fit$AllError[, "nonPermute"], 1)
default_mtry <- default_fit$mtry
default_nmin <- default_fit$nmin
rm(default_fit)

mtry_grid <- c(2L, 4L, 6L, 10L, 15L, 20L)
nmin_grid <- c(1L, 5L, 10L, 20L, 30L)
tuning_grid <- expand.grid(mtry = mtry_grid, nmin = nmin_grid)
tuning_grid$oob_error <- NA_real_
best_fit <- NULL
best_error <- Inf

cat("Question 2: evaluating", nrow(tuning_grid), "tuning settings.\n")
for (setting in seq_len(nrow(tuning_grid))) {
  fit <- RLT(
    x = train_pcs,
    y = train_labels,
    model = "classification",
    ntrees = q2_tree_count,
    mtry = tuning_grid$mtry[setting],
    nmin = tuning_grid$nmin[setting],
    resample.prob = 1,
    replacement = TRUE,
    reinforcement = FALSE
  )
  setting_error <- tail(fit$AllError[, "nonPermute"], 1)
  tuning_grid$oob_error[setting] <- setting_error
  if (setting_error < best_error) {
    best_error <- setting_error
    best_fit <- fit
  }
  if (setting %% 5L == 0L || setting == nrow(tuning_grid)) {
    cat("  completed", setting, "of", nrow(tuning_grid), "settings\n")
```

```
  }
  rm(fit)
}

selected_index <- which.min(tuning_grid$oob_error)
selected_setting <- tuning_grid[selected_index, c("mtry", "nmin"), drop = FALSE]
if (is.null(best_fit) ||
    best_fit$mtry != selected_setting$mtry ||
    best_fit$nmin != selected_setting$nmin) {
  stop("The selected model was not retained correctly.")
}
selected_importance <- data.frame(
  component = colnames(train_pcs),
  oob_permutation_importance = as.numeric(best_fit$VarImp[1, ]),
  row.names = NULL
)
tuning_grid$selected <- seq_len(nrow(tuning_grid)) == selected_index
write.csv(
  tuning_grid,
  file.path(output_dir, "q2-tuning-grid.csv"),
  row.names = FALSE
)
write.csv(
  selected_importance,
  file.path(output_dir, "q2-oob-importance.csv"),
  row.names = FALSE
)
importance_plot_data <- selected_importance[
  order(selected_importance$oob_permutation_importance),
]
importance_plot_data$component <- factor(
  importance_plot_data$component,
  levels = importance_plot_data$component
)
importance_plot <- ggplot(
  importance_plot_data,
  aes(x = component, y = oob_permutation_importance)
) +
  geom_col(fill = "#0072B2", width = 0.75) +
  coord_flip() +
  labs(x = NULL, y = "OOB permutation importance") +
  theme_minimal(base_size = 10)
ggsave(
  file.path(output_dir, "q2-importance.pdf"),
  importance_plot,
  width = 6.5,
  height = 4.2,
  units = "in"
)

test_prediction_index <- predict(best_fit, test_pcs)$Prediction
test_prediction <- factor(
  levels(train_labels)[test_prediction_index + 1L],
  levels = levels(train_labels)
)
confusion <- table(Actual = test_labels, Predicted = test_prediction)
test_error <- mean(test_prediction != test_labels)
write.csv(
  as.data.frame.matrix(confusion),
  file.path(output_dir, "q2-confusion-matrix.csv")
)

q2_summary <- list(
  seed = q2_seed,
  training_n = nrow(train_pcs),
  test_n = nrow(test_pcs),
  training_class_counts = table(train_labels),
  test_class_counts = table(test_labels),
  default_mtry = default_mtry,
  default_nmin = default_nmin,
  default_oob_error = default_oob_error,
  selected_mtry = selected_setting$mtry,
  selected_nmin = selected_setting$nmin,
  selected_oob_error = best_error,
  test_error = test_error,
  confusion = confusion
)
saveRDS(
  list(
    q1_seed = q1_seed,
    q1_results = q1_results,
    q2 = q2_summary,
    tuning_grid = tuning_grid,
    importance = selected_importance
  ),
  file.path(output_dir, "homework-10-results.rds")
)

cat("\nQuestion 1 seed:", q1_seed, "\n")
print(q1_results, row.names = FALSE)
cat("\nQuestion 2 seed:", q2_seed, "\n")
cat("Training/test sizes:", nrow(train_pcs), "/", nrow(test_pcs), "\n")
cat("Default mtry/nmin:", default_mtry, "/", default_nmin,
    " OOB error:", sprintf("%.4f", default_oob_error), "\n")
cat("Selected mtry/nmin:", selected_setting$mtry, "/", selected_setting$nmin,
    " OOB error:", sprintf("%.4f", best_error), "\n")
cat("Test error:", sprintf("%.4f", test_error), "\n")
print(confusion)
cat("\nOOB permutation importance:\n")
print(selected_importance[order(-selected_importance$oob_permutation_importance), ],
      row.names = FALSE)

q2 <- q2_summary
format_rows <- function(data, columns, formatter) {
  vapply(seq_len(nrow(data)), function(i) {
    paste0(formatter(data[i, columns, drop = FALSE]), " \\\\")
  }, character(1))
}
node_rows <- format_rows(
  node_results <- q1_results[q1_results$parameter == "Node size (mtry = 3)", ],
  c("value", "squared_bias", "variance"),
  function(row) sprintf("%d & %.4f & %.4f", row$value, row$squared_bias, row$variance)
)
mtry_rows <- format_rows(
  mtry_results <- q1_results[q1_results$parameter == "mtry (node size = 10)", ],
  c("value", "squared_bias", "variance"),
```

```
  function(row) sprintf("%d & %.4f & %.4f", row$value, row$squared_bias, row$variance)
)
grid_rows <- format_rows(
  tuning_grid,
  c("mtry", "nmin", "oob_error", "selected"),
  function(row) sprintf(
    "%d & %d & %.4f & %s",
    row$mtry, row$nmin, row$oob_error,
    if (row$selected) "Yes" else ""
  )
)
importance_order <- order(-selected_importance$oob_permutation_importance)
importance_rows <- format_rows(
  selected_importance[importance_order, ],
  c("component", "oob_permutation_importance"),
  function(row) sprintf(
    "%s & %.4f",
    row$component, row$oob_permutation_importance
  )
)
confusion_rows <- vapply(seq_len(nrow(confusion)), function(i) {
  sprintf(
    "%s & %d & %d & %d \\\\",
    rownames(confusion)[i],
    confusion[i, 1],
    confusion[i, 2],
    confusion[i, 3]
  )
}, character(1))

node_1 <- node_results[node_results$value == 1, ]
node_5 <- node_results[node_results$value == 5, ]
node_30 <- node_results[node_results$value == 30, ]
mtry_1 <- mtry_results[mtry_results$value == 1, ]
mtry_5 <- mtry_results[mtry_results$value == 5, ]
class_counts <- function(counts) {
  paste(
    sprintf("%s: %d", names(counts), as.integer(counts)),
    collapse = ", "
  )
}
test_mistakes <- sum(confusion) - sum(diag(confusion))
latex_file <- file.path(output_dir, "homework-10.tex")
latex_lines <- c(
  "\\documentclass[10pt]{article}",
  "\\usepackage[margin=0.65in]{geometry}",
  "\\usepackage{graphicx,booktabs,longtable,fancyvrb}",
  "\\setlength{\\parindent}{0pt}",
  "\\setlength{\\parskip}{4pt}",
  "\\pagestyle{plain}",
  "\\begin{document}",
  "\\begin{center}{\\LARGE\\bfseries Homework 10: Random Forests}\\end{center}",
  "\\section*{Question 1: Bias-variance trade-off simulation}",
  sprintf(
    paste0(
      "The simulation used 100 training sets of 200 observations and the same ",
      "200 test predictors throughout. Predictors were jointly normal with ",
      "pairwise correlation 0.3; $Y=X_1+X_3+X_5+\\epsilon$, with independent ",
      "standard normal noise. Each forest had 500 trees, each fit sampled ",
      "200 observations with replacement, and all settings within each ",
      "repetition used the same training data. Seed: %d. Squared bias was ",
      "estimated at each test point from the squared difference between the ",
      "average prediction and the true function; variance was estimated ",
      "across repetitions. Both are averaged over 200 test points."
    ),
    q1_seed
  ),
  "\\subsection*{Varying node size with $mtry=3$}",
  "\\begin{center}",
  "\\begin{tabular}{rrr}",
  "\\toprule Node size & Averaged squared bias & Averaged variance \\\\",
  "\\midrule",
  node_rows,
  "\\bottomrule",
  "\\end{tabular}",
  "\\end{center}",
  "\\begin{center}",
  "\\includegraphics[width=0.78\\linewidth]{q1-node-size.pdf}",
  "\\end{center}",
  sprintf(
    paste0(
      "Node size 30 substantially smooths the fit: squared bias rises from ",
      "%.3f at size 1 to %.3f, while variance falls from %.3f to %.3f. ",
      "Because the true regression function is additive and smooth, large ",
      "leaves underfit its variation. Among tested sizes, size 5 has the ",
      "smallest estimated squared-bias-plus-variance total (%.3f)."
    ),
    node_1$squared_bias, node_30$squared_bias, node_1$variance,
    node_30$variance, node_5$squared_bias + node_5$variance
  ),
  "\\subsection*{Varying $mtry$ with node size 10}",
  "\\begin{center}",
  "\\begin{tabular}{rrr}",
  "\\toprule $mtry$ & Averaged squared bias & Averaged variance \\\\",
  "\\midrule",
  mtry_rows,
  "\\bottomrule",
  "\\end{tabular}",
  "\\end{center}",
  "\\begin{center}",
  "\\includegraphics[width=0.78\\linewidth]{q1-mtry.pdf}",
  "\\end{center}",
  sprintf(
    paste0(
      "Increasing $mtry$ makes informative predictors more likely to be ",
      "available at a split, reducing squared bias from %.3f at $mtry=1$ ",
      "to %.3f at $mtry=5$. The trade-off is greater variance (%.3f to ",
      "%.3f), as trees use more similar candidate sets and become less ",
      "diverse. $mtry=5$ had the smallest estimated squared-bias-plus-",
      "variance among those tested; the variance cost may dominate in ",
      "other datasets."
    ),
    mtry_1$squared_bias, mtry_5$squared_bias, mtry_1$variance, mtry_5$variance
  ),
  paste0(
    "With 100 rather than 200 bootstrap draws, I would initially expect a ",
```

```
  "shift toward smaller node sizes to preserve comparable tree resolution: ",
  "the expected number of distinct observations is about 63 rather than ",
  "127. However, smaller samples also make trees noisier, which can favor ",
  "larger leaves. This direction is a hypothesis to check by simulation, ",
  "not a guaranteed result."
),
"\\section*{Question 2: Tuning random forests for MNIST}",
sprintf(
  paste0(
    "The first 1,000 observations supplied training data and the remaining ",
    "1,000 supplied held-out test data; only digits 1, 6, and 7 were kept. ",
    "PCA was centered but not scaled and fitted only to training pixels. ",
    "The training set had %d observations (%s); the test set had %d ",
    "observations (%s). Each forest had 1,000 trees and bootstrap samples ",
    "of the training-set size. Seed: %d."
  ),
  q2$training_n, class_counts(q2$training_class_counts),
  q2$test_n, class_counts(q2$test_class_counts), q2$seed
),
"\\subsection*{Default model and OOB tuning}",
sprintf(
  paste0(
    "The default model used $mtry=%d$ and node size %d; its OOB ",
    "misclassification rate was \\textbf{%.4f}. The grid minimum was ",
    "$mtry=%d$ and node size %d, with OOB error ",
    "\\textbf{%.4f}. OOB error was used for selection."
  ),
  q2$default_mtry, q2$default_nmin, q2$default_oob_error,
  q2$selected_mtry, q2$selected_nmin, q2$selected_oob_error
),
"\\begin{longtable}{rrrc}",
"\\caption{OOB misclassification rate for the tuning grid.}\\\\",
"\\toprule $mtry$ & Node size & OOB error & Selected \\\\",
"\\midrule",
"\\endfirsthead",
"\\toprule $mtry$ & Node size & OOB error & Selected \\\\",
"\\midrule",
"\\endhead",
grid_rows,
"\\bottomrule",
"\\end{longtable}",
"\\subsection*{Selected-model permutation importance}",
paste0(
  "RLT's OOB permutation importance is reported for all 20 principal ",
  "components; larger values indicate greater importance under this ",
  "measure. PC2 and PC1 are the most influential by a wide margin. ",
  "Principal components combine pixels, so their importance does not ",
  "identify individual image pixels."
),
"\\begin{center}",
"\\includegraphics[width=0.78\\linewidth]{q2-importance.pdf}",
"\\end{center}",
"\\begin{center}",
"\\begin{tabular}{lr}",
"\\toprule Component & OOB permutation importance \\\\",
"\\midrule",
importance_rows,
"\\bottomrule",
"\\end{tabular}",
"\\end{center}",
"\\subsection*{Held-out test assessment}",
sprintf(
  paste0(
    "The selected forest misclassified %d of %d test observations, a ",
    "misclassification rate of \\textbf{%.4f}. Test labels were not used ",
    "to fit PCA or select the model. Nearly all held-out digits were ",
    "classified correctly; digit 7 had the most mistakes."
  ),
  test_mistakes, q2$test_n, q2$test_error
),
"\\begin{center}",
"\\begin{tabular}{rrrr}",
"\\toprule Actual & Predicted 1 & Predicted 6 & Predicted 7 \\\\",
"\\midrule",
confusion_rows,
"\\bottomrule",
"\\end{tabular}",
"\\end{center}",
"\\section*{Question 3: Planning a random-forest analysis skill}",
"\\subsection*{(a) Skill design}",
paste0(
  "I would create a brief \\texttt{SKILL.md} with YAML frontmatter ",
  "containing a short name and description, followed by a focused ",
  "instruction body for future, unspecified datasets. I would not restate ",
  "routine random-forest knowledge that a capable AI can apply reliably; ",
  "instead, I would emphasize personal workflow and structural preferences, ",
  "while retaining essential reproducibility and data-suitability checks. ",
  "The skill would specify a dataset-aware tuning grid, prefer OOB ",
  "evaluation or state when cross-validation is appropriate, and set strict ",
  "table, figure, and report requirements. It would remain under 50 lines, ",
  "counting blank lines, so its guidance remains concise."
),
"\\subsection*{(b) Ten instructions, from basic to personal}",
"\\begin{enumerate}",
paste0(
  "\\item Set and report a fixed random seed before any random sampling ",
  "or fitting."
),
paste0(
  "\\item Verify response type, predictor encoding, dimensions, missingness, ",
  "and other data requirements before fitting; explain any required changes."
),
paste0(
  "\\item Fit a baseline random forest with 500 trees and the package's ",
  "default $mtry$ and node-size settings."
),
paste0(
  "\\item Define and report an explicit, data-appropriate tuning grid, ",
  "focusing on $mtry$ and node size (minimum leaf size)."
),
paste0(
  "\\item Use OOB error as the primary tuning criterion; use repeated ",
  "cross-validation when OOB is unsuitable, and exclude test data from ",
  "model selection."
),
paste0(
```

```
    "\\item Select task-appropriate metrics: for classification, report ",
    "OOB/test error and a confusion matrix (and ROC-AUC when appropriate); ",
    "for regression, report RMSE and $R^2$."
  ),
  paste0(
    "\\item Include a compact, clearly labeled table comparing every tuning ",
    "setting and its validation performance, with the selected setting marked."
  ),
  paste0(
    "\\item Include one publication-ready diagnostic figure, such as OOB ",
    "error versus the number of trees or a variable-importance plot, with ",
    "labeled axes."
  ),
  paste0(
    "\\item Interpret predictive performance, limitations, and the top three ",
    "to five influential predictors without overstating importance."
  ),
  paste0(
    "\\item Keep the final report to one printed page at 11-point font; ",
    "prioritize concise evidence, consistent table/figure formatting, and ",
    "no filler."
  ),
  "\\end{enumerate}",
  "\\clearpage",
  "\\section*{Appendix: Reproducible R code}",
  paste0(
    "The script generates the simulations, tuning results, importance ",
    "estimates, figures, and this report. It expects the compressed MNIST ",
    "file in the homework folder:\\\\",
    "\\texttt{data/}\\texttt{mnist-first2000.csv.gz}."
  ),
  "\\VerbatimInput[fontsize=\\tiny]{homework-10.R}",
  "\\end{document}"
)
writeLines(latex_lines, latex_file)

if (!nzchar(Sys.which("xelatex"))) {
  stop("xelatex is required to build homework-10.pdf.")
}
old_working_directory <- setwd(output_dir)
latex_output <- system2(
  "xelatex",
  c("-interaction=nonstopmode", "-halt-on-error", "homework-10.tex"),
  stdout = TRUE,
  stderr = TRUE
)
setwd(old_working_directory)
if (!is.null(attr(latex_output, "status")) &&
    attr(latex_output, "status") != 0L) {
  cat(latex_output, sep = "\n")
  stop("xelatex failed; see the error output above.")
}
if (!file.exists(file.path(output_dir, "homework-10.pdf"))) {
  stop("xelatex finished without producing homework-10.pdf.")
}
cat("\nWrote report:", file.path(output_dir, "homework-10.pdf"), "\n")
```